

## Chapter 9. Dynamic programming



### In this chapter

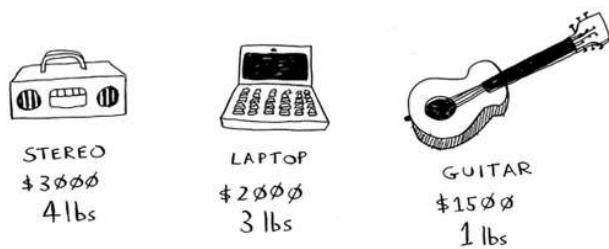
- You learn dynamic programming, a technique to solve a hard problem by breaking it up into subproblems and solving those subproblems first.
- Using examples, you learn to how to come up with a dynamic programming solution to a new problem.

### The knapsack problem



Let's revisit the knapsack problem from [chapter 8](#). You're a thief with a knapsack that can carry 4 lb of goods.

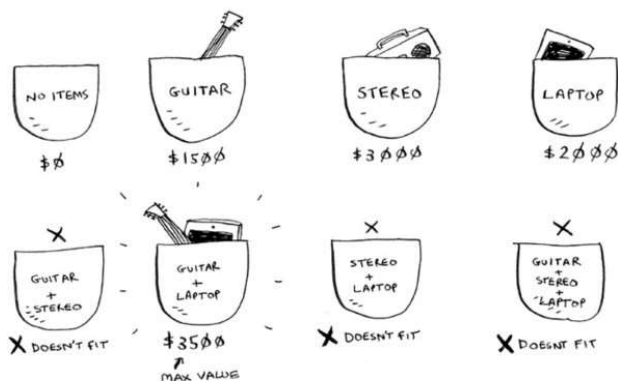
You have three items that you can put into the knapsack.



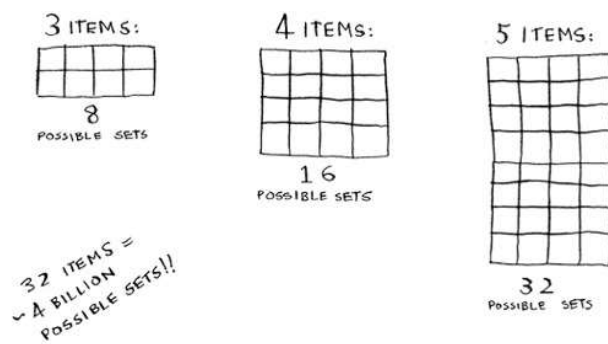
What items should you steal so that you steal the maximum money's worth of goods?

### The simple solution

The simplest algorithm is this: you try every possible set of goods and find the set that gives you the most value.



This works, but it's really slow. For 3 items, you have to calculate 8 possible sets. For 4 items, you have to calculate 16 sets. With every item you add, the number of sets you have to calculate doubles! This algorithm takes  $O(2^n)$  time, which is very, very slow.



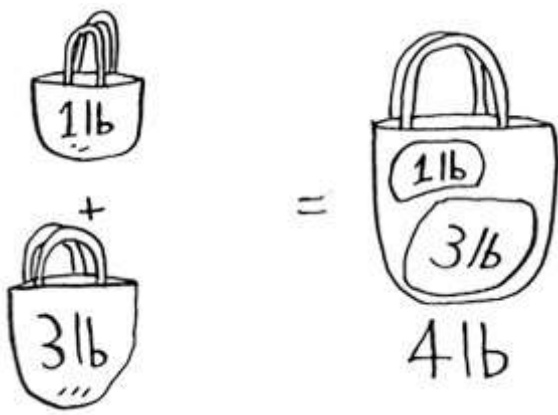
That's impractical for any reasonable number of goods. In [chapter 8](#), you saw how to calculate an *approximate* solution. That solution will be close to the optimal solution, but it may not be the optimal solution.

So how do you calculate the optimal solution?

## Dynamic programming

Answer: With dynamic programming! Let's see how the dynamic-programming algorithm works here. Dynamic programming starts by solving sub-problems and builds up to solving the big problem.

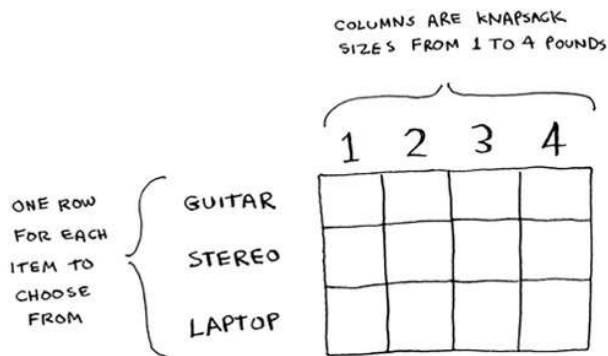
For the knapsack problem, you'll start by solving the problem for smaller knapsacks (or "sub-knapsacks") and then work up to solving the original problem.



*Dynamic programming is a hard concept, so don't worry if you don't get it right away. We're going to look at a lot of examples.*

I'll start by showing you the algorithm in action first. After you've seen it in action once, you'll have a lot of questions! I'll do my best to address every question.

Every dynamic-programming algorithm starts with a grid. Here's a grid for the knapsack problem.



The rows of the grid are the items, and the columns are knapsack weights from 1 lb to 4 lb. You need all of those columns because they will help you calculate the values of the sub-knapsacks.

The grid starts out empty. You're going to fill in each cell of the grid. Once the grid is filled in, you'll have your answer to this problem! Please follow along. Make your own grid, and we'll fill it out together.

### The guitar row

I'll show you the exact formula for calculating this grid later. Let's do a walk-through first. Start with the first row.

|        | 1 | 2 | 3 | 4 |
|--------|---|---|---|---|
| GUITAR |   |   |   |   |
| STEREO |   |   |   |   |
| LAPTOP |   |   |   |   |

This is the *guitar* row, which means you're trying to fit the guitar into the knapsack. At each cell, there's a simple decision: do you steal the guitar or not? Remember, you're trying to find the set of items to steal that will give you the most value.

The first cell has a knapsack of capacity 1 lb. The guitar is also 1 lb, which means it fits into the knapsack! So the value of this cell is \$1,500, and it contains a guitar.

Let's start filling in the grid.

|        | 1           | 2 | 3 | 4 |
|--------|-------------|---|---|---|
| GUITAR | \$1500<br>G |   |   |   |
| STEREO |             |   |   |   |
| LAPTOP |             |   |   |   |

Like this, each cell in the grid will contain a list of all the items that fit into the knapsack at that point.

Let's look at the next cell. Here you have a knapsack of capacity 2 lb. Well, the guitar will definitely fit in there!

|        | 1           | 2           | 3 | 4 |
|--------|-------------|-------------|---|---|
| GUITAR | \$1500<br>G | \$1500<br>G |   |   |
| STEREO |             |             |   |   |
| LAPTOP |             |             |   |   |

The same for the rest of the cells in this row. Remember, this is the first row, so you have *only* the guitar to choose from. You're pretending that the other two items aren't available to steal right now.

|        | 1           | 2           | 3           | 4           |
|--------|-------------|-------------|-------------|-------------|
| GUITAR | \$1500<br>G | \$1500<br>G | \$1500<br>G | \$1500<br>G |
| STEREO |             |             |             |             |
| LAPTOP |             |             |             |             |

At this point, you're probably confused. Why are you doing this for knapsacks with a capacity of 1 lb, 2 lb, and so on, when the problem talks about a 4 lb knapsack? Remember how I told you that dynamic programming starts with a small problem and builds up to the big problem? You're solving subproblems here that will help you to solve the big problem. Read on, and things will become clearer.

At this point, your grid should look like this.

|        | 1           | 2           | 3           | 4           |
|--------|-------------|-------------|-------------|-------------|
| GUITAR | \$1500<br>G | \$1500<br>G | \$1500<br>G | \$1500<br>G |
| STEREO |             |             |             |             |
| LAPTOP |             |             |             |             |

Remember, you're trying to maximize the value of the knapsack. *This row represents the current best guess for this max.* So right now, according to this row, if you had a knapsack of capacity 4 lb, the max value you could put in there would be \$1,500.

|        | 1           | 2           | 3           | 4           |
|--------|-------------|-------------|-------------|-------------|
| GUITAR | \$1500<br>G | \$1500<br>G | \$1500<br>G | \$1500<br>G |
| STEREO |             |             |             |             |
| LAPTOP |             |             |             |             |

← OUR CURRENT BEST GUESS FOR WHAT THE THIEF SHOULD STEAL: THE GUITAR FOR \$1500

You know that's not the final solution.  
As we go through the algorithm, you'll refine your estimate.

### The stereo row

Let's do the next row. This one is for the stereo. Now that you're on the second row, you can steal the stereo or the guitar. At every row, you can steal the item at that row or the items in the rows above it. So you can't choose to steal the laptop right now, but you can steal the stereo and/or the guitar. Let's start with the first cell, a knapsack of capacity 1 lb. The current max value you can fit into a knapsack of 1 lb is \$1,500.

|        | 1           | 2           | 3           | 4           |
|--------|-------------|-------------|-------------|-------------|
| GUITAR | \$1500<br>G | \$1500<br>G | \$1500<br>G | \$1500<br>G |
| STEREO |             |             |             |             |
| LAPTOP |             |             |             |             |

Should you steal the stereo or not?

You have a knapsack of capacity 1 lb.  
Will the stereo fit in there? Nope, it's too heavy! Because you can't fit the stereo, \$1,500 *remains* the max guess for a 1 lb knapsack.



|        | 1                | 2           | 3           | 4           |
|--------|------------------|-------------|-------------|-------------|
| GUITAR | \$1500<br>↓<br>G | \$1500<br>G | \$1500<br>G | \$1500<br>G |
| STEREO | \$1500<br>G      |             |             |             |
| LAPTOP |                  |             |             |             |

Same thing for the next two cells. These knapsacks have a capacity of 2 lb and 3 lb. The old max value for both was \$1,500.

|        | 1                | 2                | 3                | 4           |
|--------|------------------|------------------|------------------|-------------|
| GUITAR | \$1500<br>↓<br>G | \$1500<br>↓<br>G | \$1500<br>↓<br>G | \$1500<br>G |
| STEREO | \$1500<br>G      | \$1500<br>G      | \$1500<br>G      |             |
| LAPTOP |                  |                  |                  |             |

The stereo still doesn't fit, so your guesses remain unchanged.

What if you have a knapsack of capacity 4 lb? Aha: the stereo finally fits! The old max value was \$1,500, but if you put the stereo in there instead, the value is \$3,000! Let's take the stereo.

|        | 1                | 2                | 3                | 4           |
|--------|------------------|------------------|------------------|-------------|
| GUITAR | \$1500<br>↓<br>G | \$1500<br>↓<br>G | \$1500<br>↓<br>G | \$1500<br>G |
| STEREO | \$1500<br>G      | \$1500<br>G      | \$1500<br>G      | \$3000<br>S |
| LAPTOP |                  |                  |                  |             |

You just updated your estimate! If you have a 4 lb knapsack, you can fit at least \$3,000 worth of goods in it. You can see from the grid that you're incrementally updating your estimate.

|        | 1           | 2           | 3           | 4           |                  |
|--------|-------------|-------------|-------------|-------------|------------------|
| GUITAR | \$1500<br>G | \$1500<br>G | \$1500<br>G | \$1500<br>G | ← OLD ESTIMATE   |
| STEREO | \$1500<br>G | \$1500<br>G | \$1500<br>G | \$3000<br>S | ← NEW ESTIMATE   |
| LAPTOP |             |             |             |             | ← FINAL SOLUTION |

## The laptop row

Let's do the same thing with the laptop! The laptop weighs 3 lb, so it won't fit into a 1 lb or a 2 lb knapsack. The estimate for the first two cells stays at \$1,500.

|        | 1           | 2           | 3           | 4           |
|--------|-------------|-------------|-------------|-------------|
| GUITAR | \$1500<br>G | \$1500<br>G | \$1500<br>G | \$1500<br>G |
| STEREO | \$1500<br>G | \$1500<br>G | \$1500<br>G | \$3000<br>S |
| LAPTOP | \$1500<br>G | \$1500<br>G |             |             |

At 3 lb, the old estimate was \$1,500. But you can choose the laptop instead, and that's worth \$2,000. So the new max estimate is \$2,000!

|        | 1                | 2                | 3                | 4                |
|--------|------------------|------------------|------------------|------------------|
| GUITAR | \$1500<br>↓<br>G | \$1500<br>↓<br>G | \$1500<br>↓<br>G | \$1500<br>↓<br>G |
| STEREO | \$1500<br>↓<br>G | \$1500<br>↓<br>G | \$1500<br>↓<br>G | \$3000<br>↓<br>S |
| LAPTOP | \$1500<br>↓<br>G | \$1500<br>↓<br>G | \$2000<br>↓<br>L |                  |

At 4 lb, things get really interesting.  
This is an important part. The current estimate is \$3,000. You can put the laptop in the knapsack, but it's only worth \$2,000.

**\$3000** vs **\$2000**  
STEREO LAPTOP

Hmm, that's not as good as the old estimate. But wait! The laptop weighs only 3 lb, so you have 1 lb free! You could put something in this 1 lb.

**\$3000** vs  $\left( \begin{matrix} \$2000 \\ \text{LAPTOP} \end{matrix} + \frac{???}{1 \text{ LB OF FREE SPACE}} \right)$

What's the maximum value you can fit into 1 lb of space? Well, you've been calculating it all along.

MAX VALUE FOR 1 lb →

|        | 1                | 2                | 3                | 4                |
|--------|------------------|------------------|------------------|------------------|
| GUITAR | \$1500<br>↓<br>G | \$1500<br>↓<br>G | \$1500<br>↓<br>G | \$1500<br>↓<br>G |
| STEREO | \$1500<br>↓<br>G | \$1500<br>↓<br>G | \$1500<br>↓<br>G | \$3000<br>↓<br>S |
| LAPTOP | \$1500<br>↓<br>G | \$1500<br>↓<br>G | \$2000<br>↓<br>L |                  |

According to the last best estimate, you can fit the guitar into that 1 lb space, and that's worth \$1,500. So the real comparison is as follows.

$$\text{\$3000}_{\text{STEREO}} \text{ vs } \left( \text{\$2000}_{\text{LAPTOP}} + \text{\$1500}_{\text{GUITAR}} \right)$$

You might have been wondering why you were calculating max values for smaller knapsacks. I hope now it makes sense! When you have space left over, you can use the answers to those sub-problems to figure out what will fit in that space. It's better to take the laptop + the guitar for \$3,500.

The final grid looks like this.

|        | 1             | 2             | 3             | 4             |
|--------|---------------|---------------|---------------|---------------|
| GUITAR | \$1500<br>↓ G | \$1500<br>↓ G | \$1500<br>↓ G | \$1500<br>G   |
| STEREO | \$1500<br>↓ G | \$1500<br>↓ G | \$1500<br>G   | \$3000<br>S   |
| LAPTOP | \$1500<br>G   | \$1500<br>G   | \$2000<br>L   | \$3500<br>L G |

↑  
THE ANSWER!

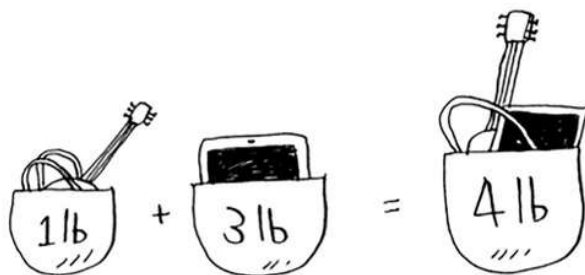
There's the answer: the maximum value that will fit in the knapsack is \$3,500, made up of a guitar and a laptop!

Maybe you think that I used a different formula to calculate the value of that last cell. That's because I skipped some unnecessary complexity when filling in the values of the earlier cells. Each cell's

value gets calculated with the same formula. Here it is.

$$\begin{array}{c} \text{ROW} \quad \text{COLUMN} \\ \downarrow \quad \downarrow \\ \text{CELL}[i][j] = \max \text{ of } \left\{ \begin{array}{l} 1. \text{ THE PREVIOUS MAX (VALUE AT CELL } [i-1][j]) \\ \text{VS} \\ 2. \text{ VALUE OF CURRENT ITEM + VALUE OF THE REMAINING SPACE} \\ \quad \quad \quad \uparrow \\ \quad \quad \quad \text{CELL}[i-1][j] - \text{ITEM'S WEIGHT} \end{array} \right\} \end{array}$$

You can use this formula with every cell in this grid, and you should end up with the same grid I did. Remember how I talked about solving subproblems? You combined the solutions to two subproblems to solve the bigger problem.



## Knapsack problem FAQ

Maybe this still feels like magic. This section answers some common questions.

### What happens if you add an item?



Suppose you realize there's a fourth item you can steal that you didn't notice

before! You can also steal an iPhone.

Do you have to recalculate everything to account for this new item? Nope.

Remember, dynamic programming keeps progressively building on your estimate. So far, these are the max values.

|        | 1           | 2           | 3           | 4            |
|--------|-------------|-------------|-------------|--------------|
| GUITAR | \$1500<br>G | \$1500<br>G | \$1500<br>G | \$1500<br>G  |
| STEREO | \$1500<br>G | \$1500<br>G | \$1500<br>G | \$3000<br>S  |
| LAPTOP | \$1500<br>G | \$1500<br>G | \$2000<br>L | \$3500<br>LG |

That means for a 4 lb knapsack, you can steal \$3,500 worth of goods. You thought that was the final max value. But let's add a row for the iPhone.

|        | 1           | 2           | 3           | 4            |
|--------|-------------|-------------|-------------|--------------|
| GUITAR | \$1500<br>G | \$1500<br>G | \$1500<br>G | \$1500<br>G  |
| STEREO | \$1500<br>G | \$1500<br>G | \$1500<br>G | \$3000<br>S  |
| LAPTOP | \$1500<br>G | \$1500<br>G | \$2000<br>L | \$3500<br>LG |
| IPHONE |             |             |             |              |

↑  
NEW ANSWER

Turns out you have a new max value! Try to fill in this new row before reading on.

Let's start with the first cell. The iPhone fits into the 1 lb knapsack. The old max

was \$1,500, but the iPhone is worth \$2,000. Let's take the iPhone instead.

|        | 1           | 2           | 3           | 4            |
|--------|-------------|-------------|-------------|--------------|
| GUITAR | \$1500<br>G | \$1500<br>G | \$1500<br>G | \$1500<br>G  |
| STEREO | \$1500<br>G | \$1500<br>G | \$1500<br>G | \$2000<br>S  |
| LAPTOP | \$1500<br>G | \$1500<br>G | \$2000<br>L | \$3500<br>LG |
| IPHONE | \$2000<br>I |             |             |              |

In the next cell, you can fit the iPhone *and* the guitar.

|             |              |             |              |
|-------------|--------------|-------------|--------------|
| \$1500<br>G | \$1500<br>G  | \$1500<br>G | \$1500<br>G  |
| \$1500<br>G | \$1500<br>G  | \$1500<br>G | \$3000<br>S  |
| \$1500<br>G | \$1500<br>G  | \$2000<br>L | \$3500<br>LG |
| \$2000<br>I | \$3500<br>IG |             |              |

For cell 3, you can't do better than take the iPhone and the guitar again, so leave it as is.

For the last cell, things get interesting. The current max is \$3,500. You can steal the iPhone instead, and you have 3 lb of space left over.

$$\begin{array}{l} \$3500 \\ \text{LAPTOP + GUITAR} \end{array} \quad \text{vs} \quad \left( \begin{array}{c} \$2000 \\ \text{IPHONE} \end{array} + \frac{???}{3 \text{ LBS FREE}} \right)$$

Those 3 lb are worth \$2,000! \$2,000  
 from the iPhone + \$2,000 from the old  
 subproblem: that's \$4,000. A new max!

Here's the new final grid.

|             |              |              |              |
|-------------|--------------|--------------|--------------|
| \$1500<br>G | \$1500<br>G  | \$1500<br>G  | \$1500<br>G  |
| \$1500<br>G | \$1500<br>G  | \$1500<br>G  | \$3000<br>S  |
| \$1500<br>G | \$1500<br>G  | \$2000<br>L  | \$3500<br>LG |
| \$2000<br>I | \$3500<br>IG | \$3500<br>IG | \$4000<br>IL |

↑  
NEW  
ANSWER

Question: Would the value of a column  
 ever go *down*? Is this possible?

|                                              |        |        |        |        |
|----------------------------------------------|--------|--------|--------|--------|
|                                              | 1      | 2      | 3      | 4      |
| MAX VALUE<br>DECREASED<br>AS WE<br>WENT ON ↓ | \$1500 | \$1500 | \$1500 | \$1500 |
|                                              | ∅      | ∅      | ∅      | \$3000 |
|                                              |        |        |        |        |

Think of an answer before reading on.

Answer: No. At every iteration, you're  
 storing the current max estimate. The  
 estimate can never get worse than it  
 was before!

## Exercise



Suppose you can steal another item: an MP3 player. It weighs 1 lb and is worth \$1,000. Should you steal it?

### What happens if you change the order of the rows?

Does the answer change? Suppose you fill the rows in this order: stereo, laptop, guitar. What does the grid look like? Fill out the grid for yourself before moving on.

Here's what the grid looks like.

|        | 1           | 2           | 3           | 4            |
|--------|-------------|-------------|-------------|--------------|
| STEREO | ∅           | ∅           | ∅           | \$3000<br>S  |
| LAPTOP | ∅           | ∅           | \$2000<br>L | \$3000<br>S  |
| GUITAR | \$1500<br>G | \$1500<br>G | \$2000<br>L | \$3500<br>LG |

The answer doesn't change. The order of the rows doesn't matter.

### Can you fill in the grid column-wise instead of row-wise?

Try it for yourself! For this problem, it doesn't make a difference. It could make a difference for other problems.

### What happens if you add a smaller item?

Suppose you can steal a necklace. It weighs 0.5 lb, and it's worth \$1,000. So far, your grid assumes that all weights

are integers. Now you decide to steal the necklace. You have 3.5 lb left over. What's the max value you can fit in 3.5 lb? You don't know! You only calculated values for 1 lb, 2 lb, 3 lb, and 4 lb knapsacks. You need to know the value of a 3.5 lb knapsack.

*Because of the necklace, you have to account for finer granularity, so the grid has to change.*

|         | 0.5 | 1 | 1.5 | 2 | 2.5 | 3 | 3.5 | 4 |
|---------|-----|---|-----|---|-----|---|-----|---|
| GUITAR  |     |   |     |   |     |   |     |   |
| STEREO  |     |   |     |   |     |   |     |   |
| LAPTOP  |     |   |     |   |     |   |     |   |
| JEWELRY |     |   |     |   |     |   |     |   |

### Can you steal fractions of an item?

Suppose you're a thief in a grocery store. You can steal bags of lentils and rice. If a whole bag doesn't fit, you can open it and take as much as you can carry. So now it's not all or nothing—you can take a fraction of an item. How do you handle this using dynamic programming?

Answer: You can't. With the dynamic-programming solution, you either take the item or not. There's no way for it to figure out that you should take half an item.

But this case is also easily solved using a greedy algorithm! First, take as much as you can of the most valuable item.

When that runs out, take as much as you can of the next most valuable item, and so on.

For example, suppose you have these items to choose from.



Quinoa is more expensive per pound than anything else. So, take all the quinoa you can carry! If that fills your knapsack, that's the best you can do.

If the quinoa runs out and you still have space in your knapsack, take the next most valuable item, and so on.



## Optimizing your travel itinerary

Suppose you're going to London for a nice vacation. You have two days there and a lot of things you want to do. You can't do everything, so you make a list.

| ATTRACTION           | TIME              | RATING |
|----------------------|-------------------|--------|
| WESTMINSTER ABBEY    | $\frac{1}{2}$ DAY | 7      |
| GLOBE THEATER        | $\frac{1}{2}$ DAY | 6      |
| NATIONAL GALLERY     | 1 DAY             | 9      |
| BRITISH MUSEUM       | 2 DAYS            | 9      |
| ST. PAUL'S CATHEDRAL | $\frac{1}{2}$ DAY | 8      |

For each thing you want to see, you write down how long it will take and rate how much you want to see it. Can you figure out what you should see, based on this list?

It's the knapsack problem again! Instead of a knapsack, you have a limited amount of time. And instead of stereos and laptops, you have a list of places you want to go. Draw the dynamic-programming grid for this list before moving on.

Here's what the grid looks like.

|                  | $\frac{1}{2}$ | 1 | $1\frac{1}{2}$ | 2 |
|------------------|---------------|---|----------------|---|
| WESTMINSTER      |               |   |                |   |
| GLOBE THEATRE    |               |   |                |   |
| NATIONAL GALLERY |               |   |                |   |
| BRITISH MUSEUM   |               |   |                |   |
| ST. PAUL'S       |               |   |                |   |

Did you get it right? Fill in the grid.  
 What places should you end up seeing?  
 Here's the answer.

|                  | $\frac{1}{2}$  | 1                | $1\frac{1}{2}$    | 2                 |
|------------------|----------------|------------------|-------------------|-------------------|
| WESTMINSTER      | 7 <sub>W</sub> | 7 <sub>W</sub>   | 7 <sub>W</sub>    | 7 <sub>W</sub>    |
| GLOBE THEATRE    | 7 <sub>W</sub> | 13 <sub>WG</sub> | 13 <sub>WG</sub>  | 13 <sub>WG</sub>  |
| NATIONAL GALLERY | 7 <sub>W</sub> | 13 <sub>WG</sub> | 16 <sub>WN</sub>  | 22 <sub>WGN</sub> |
| BRITISH MUSEUM   | 7 <sub>W</sub> | 13 <sub>WG</sub> | 16 <sub>WN</sub>  | 22 <sub>WGN</sub> |
| ST PAUL'S        | 8 <sub>S</sub> | 15 <sub>WS</sub> | 21 <sub>WGS</sub> | 24 <sub>WNS</sub> |

↑  
 FINAL ANSWER:  
 WESTMINSTER ABBEY,  
 NATIONAL GALLERY,  
 ST. PAUL'S CATHEDRAL

## Handling items that depend on each other

Suppose you want to go to Paris, so you add a couple of items on the list.

|              |                    |   |
|--------------|--------------------|---|
| EIFFEL TOWER | $1\frac{1}{2}$ DAY | 8 |
| THE LOUVRE   | $1\frac{1}{2}$ DAY | 9 |
| NOTRE DAME   | $1\frac{1}{2}$ DAY | 7 |

These places take a lot of time, because first you have to travel from London to Paris. That takes half a day. If you want to do all three items, it will take four and a half days.

Wait, that's not right. You don't have to travel to Paris for each item. Once you're in Paris, each item should only take a day. So it should be one day per item + half a day of travel = 3.5 days, not 4.5 days.

If you put the Eiffel Tower in your knapsack, then the Louvre becomes "cheaper"—it will only cost you a day instead of 1.5 days. How do you model this in dynamic programming?

You can't. Dynamic programming is powerful because it can solve subproblems and use those answers to solve the big problem. *Dynamic programming only works when each subproblem is discrete—when it doesn't depend on other subproblems.* That means there's no way to account for Paris using the dynamic-programming algorithm.

### **Is it possible that the solution will require more than two sub-knapsacks?**

It's possible that the best solution involves stealing more than two items. The way the algorithm is set up, you're combining two knapsacks at most—you'll never have more than two sub-knapsacks. But it's possible for those sub-knapsacks to have their own sub-knapsacks.



### **Is it possible that the best solution doesn't fill the knapsack completely?**

Yes. Suppose you could also steal a diamond.



DIAMOND  
\$1 MILLION DOLLARS  
3.5lbs

This is a big diamond: it weighs 3.5 pounds. It's worth a million dollars, way more than anything else. You should definitely steal it! But there's half a pound of space left, and nothing will fit in that space.

## Exercise

### 9.2

*Suppose you're going camping. You have a knapsack that will hold 6 lb, and you can take the following items. Each has a value, and the higher the value, the more important the item is:*

- Water, 3 lb, 10
- Book, 1 lb, 3
- Food, 2 lb, 9
- Jacket, 2 lb, 5
- Camera, 1 lb, 6

*What's the optimal set of items to take on your camping trip?*

## Longest common substring



You've seen one dynamic programming problem so far. What are the takeaways?

- Dynamic programming is useful *when you're trying to optimize something given a constraint*. In the knapsack problem, you had to maximize the value of the goods you stole, constrained by the size of the knapsack.
- You can use dynamic programming when the problem can be broken into discrete subproblems, and they don't depend on each other.

It can be hard to come up with a dynamic-programming solution. That's what we'll focus on in this section.

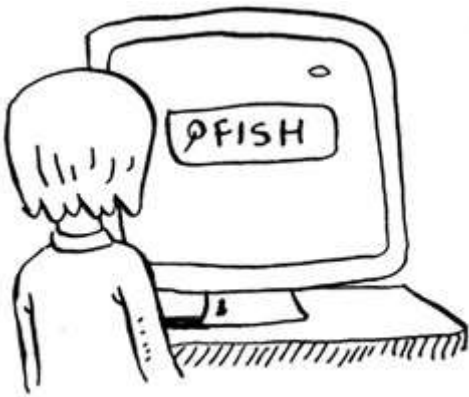
Some general tips follow:

- Every dynamic-programming solution involves a grid.
- The values in the cells are usually what you're trying to optimize. For the knapsack problem, the values were the value of the goods.
- Each cell is a subproblem, so think about how you can divide your



problem into subproblems. That will help you figure out what the axes are.

Let's look at another example. Suppose you run dictionary.com. Someone types in a word, and you give them the definition.



But if someone misspells a word, you want to be able to guess what word they meant. Alex is searching for *fish*, but he accidentally put in *hish*. That's not a word in your dictionary, but you have a list of words that are similar.

SIMILAR TO "HISH":

- FISH
- VISTA

(This is a toy example, so you'll limit your list to two words. In reality, this list would probably be thousands of words.)

Alex typed *hish*. Which word did Alex mean to type: *fish* or *vista*?

## Making the grid

What does the grid for this problem look like? You need to answer these questions:

- What are the values of the cells?
- How do you divide this problem into subproblems?
- What are the axes of the grid?

In dynamic programming, you're trying to *maximize* something. In this case, you're trying to find the longest substring that two words have in common. What substring do *hish* and *fish* have in common? How about *hish* and *vista*? That's what you want to calculate.

Remember, the values for the cells are usually what you're trying to optimize. In this case, the values will probably be a number: the length of the longest substring that the two strings have in common.

How do you divide this problem into subproblems? You could compare substrings. Instead of comparing *hish* and *fish*, you could compare *his* and *fis* first. Each cell will contain the length of the longest substring that two substrings have in common. This also gives you a clue that the axes will probably be the two words. So the grid probably looks like this.

|   | H | I | S | H |
|---|---|---|---|---|
| F |   |   |   |   |
| I |   |   |   |   |
| S |   |   |   |   |
| H |   |   |   |   |

If this seems like black magic to you, don't worry. This is hard stuff—that's why I'm teaching it so late in the book! Later, I'll give you an exercise so you can practice dynamic programming yourself.

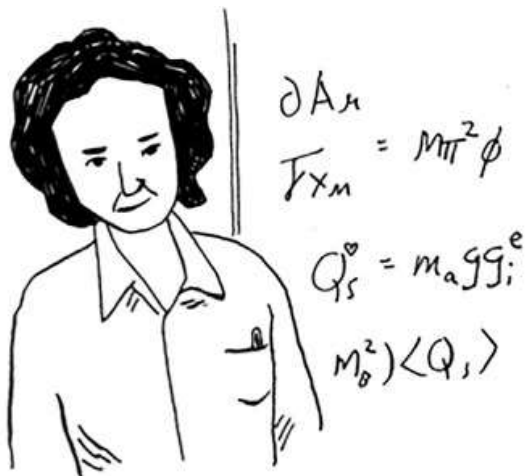
### Filling in the grid

Now you have a good idea of what the grid should look like. What's the formula for filling in each cell of the grid? You can cheat a little, because you already know what the solution should be—*hish* and *fish* have a substring of length 3 in common: *ish*.

But that still doesn't tell you the formula to use. Computer scientists sometimes joke about using the Feynman algorithm. The *Feynman algorithm* is named after the famous physicist Richard Feynman, and it works like this:

1. Write down the problem.
2. Think real hard.

3. Write down the solution.



Computer scientists are a fun bunch!

The truth is, there's no easy way to calculate the formula here. You have to experiment and try to find something that works. Sometimes algorithms aren't an exact recipe. They're a framework that you build your idea on top of.

Try to come up with a solution to this problem yourself. I'll give you a hint—part of the grid looks like this.

|   | H | I | S | H |
|---|---|---|---|---|
| F | 0 | 0 |   |   |
| I |   |   |   |   |
| S |   |   | 2 | 0 |
| H |   |   |   | 3 |

What are the other values? Remember that each cell is the value of a *subproblem*. Why does cell (3, 3) have a value of

2? Why does cell (3, 4) have a value of 0?

Read on after you've tried to come up with a formula yourself. Even if you don't get it right, my explanation will make a lot more sense.

## The solution

Here's the final grid.

|   | H | I | S | H |
|---|---|---|---|---|
| F | 0 | 0 | 0 | 0 |
| I | 0 | 1 | 0 | 0 |
| S | 0 | 0 | 2 | 0 |
| H | 1 | 0 | 0 | 3 |

Here's my formula for filling in each cell.

1. IF THE LETTERS  
DONT MATCH,  
THE VALUE IS  
ZERO.

|   | H | I | S | H |
|---|---|---|---|---|
| F | 0 | 0 | 0 | 0 |
| I | 0 | 1 | 0 | 0 |
| S | 0 | 0 | 2 | 0 |
| H | 1 | 0 | 0 | 3 |

2. IF THEY DO MATCH,  
THIS VALUE IS  
VALUE OF TOP-LEFT NEIGHBOR + 1

Here's how the formula looks in pseudocode:

```

if word_a[i] == word_b[j]:<----- The letters match.
    cell[i][j] = cell[i-1][j-1] + 1
else: <----- The letters don't match.
    cell[i][j] = 0

```

Here's the grid for *hish* vs. *vista*.

|   | V | I | S | T | A |
|---|---|---|---|---|---|
| H | 0 | 0 | 0 | 0 | 0 |
| I | 0 | 1 | 0 | 0 | 0 |
| S | 0 | 0 | 2 | 0 | 0 |
| H | 0 | 0 | 0 | 0 | 0 |

↑ FINAL ANSWER      ↑ NOT THE FINAL ANSWER

One thing to note: for this problem, the final solution may not be in the last cell! For the knapsack problem, this last cell always had the final solution. But for the longest common substring, the solution is the largest number in the grid—and it may not be the last cell.

Let's go back to the original question: which string has more in common with *hish*? *hish* and *fish* have a substring of three letters in common. *hish* and *vista* have a substring of two letters in common.

Alex probably meant to type *fish*.

## Longest common subsequence

Suppose Alex accidentally searched for *fosh*. Which word did he mean: *fish* or

*fort*?

Let's compare them using the longest-common-substring formula.

|   | F | O | S | H |
|---|---|---|---|---|
| F | 1 | 0 | 0 | 0 |
| O | 0 | 2 | 0 | 0 |
| R | 0 | 0 | 0 | 0 |
| T | 0 | 0 | 0 | 0 |

 vs 

|   | F | O | S | H |
|---|---|---|---|---|
| F | 1 | 0 | 0 | 0 |
| I | 0 | 0 | 0 | 0 |
| S | 0 | 0 | 1 | 0 |
| H | 0 | 0 | 0 | 2 |

They're both the same: two letters! But *fosh* is closer to *fish*.

F O S H  
↓ ↓ ↓  
F I S H = 3

F O S H  
↓ ↓  
F O R T = 2

You're comparing the longest common *substring*, but you really need to compare the longest common *subsequence*: the number of letters in a sequence that the two words have in common. How do you calculate the longest common subsequence?

Here's the partial grid for *fish* and *fosh*.

|   | F | O | S | H |
|---|---|---|---|---|
| F | 1 | 1 |   |   |
| I | 1 |   |   |   |
| S |   | 1 | 2 | 2 |
| H |   |   |   |   |

Can you figure out the formula for this grid? The longest common subsequence is very similar to the longest common substring, and the formulas are pretty similar, too. Try to solve it yourself—I give the answer next.

### **Longest common subsequence—solution**

Here's the final grid.

Here's my formula for filling in each cell.

And here it is in pseudocode:

Whew—you did it! This is definitely one of the toughest chapters in the book. So is dynamic programming ever really used? *Yes:*

- Biologists use the longest common subsequence to find similarities in DNA strands. They can use this to tell how similar two animals or two diseases are. The longest common subsequence is being used to find a cure for multiple sclerosis.
- Have you ever used diff (like `git diff`)? Diff tells you the differences



between two files, and it uses dynamic programming to do so.

- We talked about string similarity. *Levenshtein distance* measures how similar two strings are, and it uses dynamic programming. Levenshtein distance is used for everything from spell-check to figuring out whether a user is uploading copyrighted data.
- Have you ever used an app that does word wrap, like Microsoft Word? How does it figure out where to wrap so that the line length stays consistent? Dynamic programming!

## Exercise

### 9.3

*Draw and fill in the grid to calculate the longest common substring between blue and clues.*

## Recap

- Dynamic programming is useful when you're trying to optimize something given a constraint.
- You can use dynamic programming when the problem can be broken into discrete subproblems.
- Every dynamic-programming solution involves a grid.
- The values in the cells are usually what you're trying to optimize.
- Each cell is a subproblem, so think about how you can divide your problem into subproblems.

- There's no single formula for calculating a dynamic-programming solution.